

Phase 1G — Billing Lifecycle & Stripe Customer Portal

Phase 1G makes BaseCenter billing production-grade. It builds on the Phase 1C Stripe integration and the Phase 1F `require_active_module(slug)` guard so that **tenant access always reflects real subscription status**. Every path degrades gracefully when Stripe is not configured — the billing page renders fully, and Stripe-dependent actions return a friendly error instead of failing.

Part A — Subscription lifecycle

1. Cancel with cancel-at-period-end

- A tenant admin can cancel a paid module subscription. This calls Stripe `Subscription.modify(cancel_at_period_end=True)` and sets the local `cancel_at_period_end` flag. **The status stays active, so access continues until the paid period ends**, at which point Stripe emits `customer.subscription.deleted` and the subscription flips to `canceled` (inactive).
- A **free** subscription (no Stripe subscription) is deactivated immediately (`status = canceled`).

2. Reactivate a scheduled cancellation

- If a subscription is scheduled to cancel, the admin can reactivate it. This calls `Subscription.modify(cancel_at_period_end=False)`, clears the local flag, and (if it had been canceled locally) restores `status = active`.

3. Status model

The local `Subscription.status` vocabulary and the mapping from Stripe:

Local status	Meaning	Stripe statuses mapped
<code>active</code>	Access granted (incl. scheduled cancel)	<code>active</code> , <code>trialing</code>
<code>past_due</code>	Payment failed, access denied	<code>past_due</code> , <code>unpaid</code>
<code>canceled</code>	Ended, access denied	<code>canceled</code> , <code>incomplete_expired</code>
<code>incomplete</code>	Initial payment not completed	<code>incomplete</code>

`cancel_at_period_end` (new boolean column) tracks a scheduled cancellation independently of status, so an `active` subscription can be shown as “Cancels at period end” without losing access.

The **access guard is unchanged**: `require_active_module(slug)` grants access only when `status == "active"`. Because a scheduled cancellation keeps the status `active`, access correctly continues until the period actually ends; `past_due`, `canceled`, and `incomplete` are all denied (403).

Part B — Dunning / past-due

Expanded Stripe webhook handling with **idempotency** — every processed event id is recorded in the new `webhook_events` table and duplicate deliveries are acknowledged without re-processing.

Stripe event	Effect on local subscription
<code>invoice.payment_failed</code>	→ <code>past_due</code> (guard now denies access)
<code>invoice.paid</code> / <code>invoice.payment_succeeded</code>	recovery: <code>past_due</code> / <code>incomplete</code> → <code>active</code>
<code>customer.subscription.updated</code>	syncs mapped status, <code>current_period_end</code> , and <code>cancel_at_period_end</code>
<code>customer.subscription.deleted</code>	→ <code>canceled</code> , clears <code>cancel_at_period_end</code>
<code>checkout.session.completed</code>	activates the subscription (existing, unchanged)

A “**Payment past due — action required**” banner is surfaced in the tenant portal (billing page, and the dashboard) whenever any subscription is `past_due`, linking the admin to update their payment method.

Part C — Invoices & receipts

- A tenant-facing, **read-only** invoices table on the billing page, fetched **live from Stripe** for the tenant’s customer.
- Each row shows invoice number, date, amount + currency, status, and links to the **hosted invoice URL** (“View”) and the **invoice PDF** (“PDF”).
- Gracefully empty when Stripe is unconfigured, the tenant has no Stripe customer, or there are no invoices yet — never errors.

Part D — Stripe Customer (Billing) Portal

- Backend endpoint (`POST /api/v1/tenant/billing/portal`, tenant-admin only) ensures a Stripe customer exists for the tenant (`get_or_create_customer`) and creates a Billing Portal session, returning its URL.

- The frontend **“Manage billing”** button (Yellow #E4F222 financial CTA) redirects the admin to the portal to manage payment methods, view invoices, and update billing details, returning to `/app/billing`.
- Returns a friendly HTTP 400 when Stripe is not configured.

Files

Backend

- `app/models/subscription.py` — new `cancel_at_period_end` boolean column.
- `app/models/webhook_event.py` — new `WebhookEvent` model (unique `event_id`) for webhook idempotency (registered in `app/models/__init__.py` and `alembic/env.py`).
- `alembic/versions/e3c4d5f6a7b8_phase1g_billing_lifecycle.py` — adds `subscriptions.cancel_at_period_end` and creates `webhook_events` (down_revision `d2b3c4e5f6a7`).
- `app/services/stripe_service.py` — `map_stripe_status`, `set_cancel_at_period_end`, `list_invoices`, `create_billing_portal_session`.
- `app/services/billing_service.py` — tenant-scoped billing overview, invoices, cancel/reactivate, and portal session (all graceful when Stripe unconfigured).
- `app/api/v1/webhooks.py` — idempotency + dunning/recovery handlers + `cancel_at_period_end` sync + canonical `canceled` spelling.
- `app/schemas/billing.py` — `BillingSubscription`, `BillingOverview`, `InvoiceOut` / `InvoiceList`, `PortalSessionResponse`, `MessageResponse`.
- `app/api/v1/tenant.py` — five tenant-admin-protected billing endpoints: `GET /billing`, `GET /billing/invoices`, `POST /billing/subscriptions/{id}/cancel`, `POST /billing/subscriptions/{id}/reactivate`, `POST /billing/portal`.

Frontend

- `app/app/app/billing/page.tsx` — billing page: past-due banner, subscription list with status badges + cancel/reactivate controls, “Manage billing” button, and the live invoices table. Graceful “not configured” state.
- `app/app/app/page.tsx` — past-due banner on the tenant dashboard.
- `components/tenant/TenantShell.tsx` — admin-only “Billing” nav item.
- `lib/api.ts` — billing types (`BillingOverview`, `BillingSubscription`, `InvoiceOut`) and client functions (`getTenantBilling`, `getTenantInvoices`, `cancelTenantSubscription`, `reactivateTenantSubscription`, `createBillingPortalSession`).

Verification

- PostgreSQL running; `alembic upgrade head` applies cleanly (head `e3c4d5f6a7b8`); `subscriptions.cancel_at_period_end` column and `webhook_events` table created.

- Backend (:8000) imports and registers all five `/api/v1/tenant/billing/*` routes.
 - `npx tsc --noEmit` and `npm run build` both pass; the `/app/billing` route builds.
 - End-to-end smoke test (API):
 - Tenant signup → **billing overview loads** with Stripe unconfigured (`stripe_configured: false`), invoices return an empty list, and the portal endpoint returns a friendly **400** — full graceful degradation.
 - **Webhook transitions** (handlers driven directly against the DB):
 - `invoice.payment_failed` → `past_due` (**guard denies**);
 - `invoice.paid` → `active` (**guard allows**);
 - `customer.subscription.updated` with `cancel_at_period_end=True` keeps status `active` (**access continues**);
 - `customer.subscription.deleted` → `canceled` (**guard denies**).
 - **Idempotency**: `webhook_events` records processed event ids; duplicates are acknowledged without re-processing.
 - **Cancel/reactivate API**: free subscription cancel → `canceled` ; reactivate → `active` ; a non-owned/unknown subscription id returns **404** (tenant isolation).
-

Summary

Capability	Mechanism
Cancel (paid)	<code>cancel_at_period_end=True</code> in Stripe + local flag; stays active until period end
Cancel (free)	Immediate <code>status=canceled</code>
Reactivate	<code>cancel_at_period_end=False</code> ; restores active
Dunning	<code>invoice.payment_failed</code> → <code>past_due</code> ; guard denies
Recovery	<code>invoice.paid</code> → <code>active</code> ; guard allows
End of period	<code>customer.subscription.deleted</code> → <code>canceled</code>
Idempotency	<code>webhook_events</code> table records processed event ids
Invoices	Read-only, live from Stripe (hosted URL + PDF)
Customer Portal	Billing Portal session (tenant-admin), Yellow CTA
Graceful degradation	Every Stripe path handles unconfigured → empty/400